CHƯƠNG 5 • ỨNG DỤNG PYTHON

5.1. Lập trình giao diện GUI cơ bản bằng Tkinter
5.2. Lập trình Web cơ bản: Frontend/Backend và Flask
5.3. SQLite: kết nối, truy vấn và thao tác dữ liệu
5.4. Lập trình socket cơ bản: TCP/UDP, client/server
5.5. Phân tích dữ liệu cơ bản với NumPy, Pandas và Matplotlib

CHƯƠNG 5 • ỨNG DỤNG PYTHON

5.1. Lập trình GUI cơ bản bằng Tkinter

5.1.1 Tổng quan GUI & Tkinter • 5.1.2 Cửa sổ chính
• 5.1.3 Hệ thống Widget • 5.1.4 Quản lý bố cục
• 5.1.5 Sự kiện • 5.1.6 Hộp thoại & Ứng dụng

5.1.1. Từ dòng lệnh đến đồ họa

Giao diện CLI

Tương tác: Nhập lệnh văn bản thuần túy

Hiển thị: Màn hình đen chữ đơn sắc

Rào cản: Đòi hỏi nhớ cú pháp lệnh

Đối tượng: Lập trình viên kỹ thuật

Giao diện GUI

Tương tác: Thao tác chuột trực quan

Hiển thị: Cửa sổ nút bấm biểu tượng

Trải nghiệm: Thân thiện dễ làm quen

Đối tượng: Mọi người dùng phổ thông

5.1.1. T ổng quan th ư viện Tkinter

Tại sao ch ọn Tkinter?

- **Tích h ợp sẵn:** Có sẵn trong bộ cài Python chuẩn
- **Nền tảng Tcl/Tk:** Giao tiếp trực tiếp bộ công cụ Tk
- **Đa hệ điều hành:** Chạy tốt trên Windows macOS Linux
- **Cú pháp g ọn nh ẹ:** Khởi tạo ứng dụng chỉ vài dòng lệnh



5.1.2. Quy trình 4 b ước tạo GUI

1

Nhập thư viện

Khai báo mô-đun tkinter

2

Tạo cửa sổ gốc

Khởi tạo đối tượng Tk()

3

Gắn Widget

Đưa nút nhãn lên cửa sổ

4

Lặp sự kiện

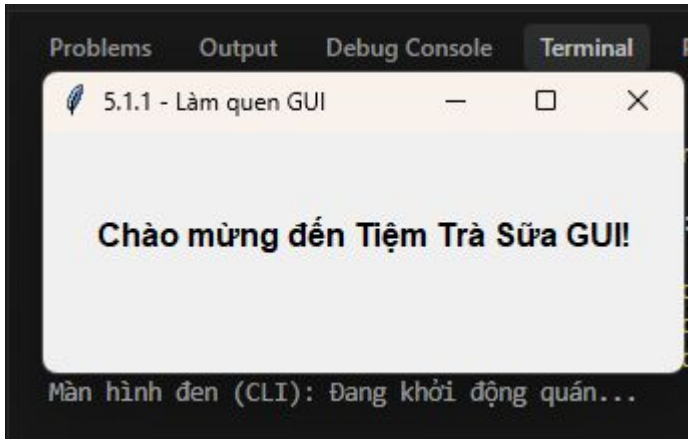
Gọi hàm mainloop() duy trì

5.1.2. Kh ởi tạo cửa sổ chính

```
import tkinter as tk

print("Màn hình đen (CLI): Đang khởi động quán...")
root = tk.Tk()
root.title("5.1.1 - Làm quen GUI")
root.geometry("320x120")

tk.Label(root, text="Chào mừng đ ế n Tiệm Trà Sữa
GUI!", font=("Arial", 12, "bold"), pady=40).pack()
root.mainloop()
```



5.1.2. Kh ởi tạo cửa sổ chính

Ý ni ệm: Xây dựng "mặt bằng" và bật nguồn hệ thống cho ứng dụng.
Các thành ph ần cốt lõi:

- root = tk.Tk(): Khởi tạo cửa sổ chính (mặt bằng ứng dụng).
- root.title("..."): Thiết lập tiêu đề cửa sổ.
- root.geometry("WxH"): Định nghĩa kích thước cửa sổ (Rộng x Cao).
- root.resizable(False, False): Cố định kích thước, ngăn người dùng kéo dãn.
- root.mainloop(): Vòng lặp sự kiện chính (**Bắt buộc** để giữ cửa sổ luôn hiển thị và lắng nghe tương tác).

5.1.3. B ốn Widget thông d ụng

Label

Hiển thị văn bản hoặc ảnh tĩnh.

Button

Nút bấm kích hoạt hàm xử lý.

Entry

Ô nhập chuỗi trên một dòng.

Text

Vùng soạn thảo văn bản nhiều dòng.

5.1.3. Widget lựa chọn và khung

Tên lớp Widget	Chức năng chính	Thuộc tính quan trọng
Frame	Khung gom nhóm thành phần con	bg, relief, padx, pady
Checkbutton	Hộp đánh dấu nhiều lựa chọn	variable, onvalue, offvalue
Radiobutton	Nút chọn một phương án duy nhất	variable, value, command
Listbox	Danh sách mục hiển thị theo dòng	selectmode, height, width
ttk.Combobox	Hộp danh sách thả xuống gọn	values, state, current()

5.1.3. Widget lựa chọn và khung

```
import tkinter as tk

root = tk.Tk()
root.title("5.1.2 - Mặt Bằng Quán Trà Sữa")
root.geometry("350x250")
root.resizable(False, False)
root.configure(bg="#fdf2e9")

root.mainloop()
```



5.1.4. Ba c ơ chế quản lý b ố cục

pack()

Xếp khối tuần tự theo hướng.

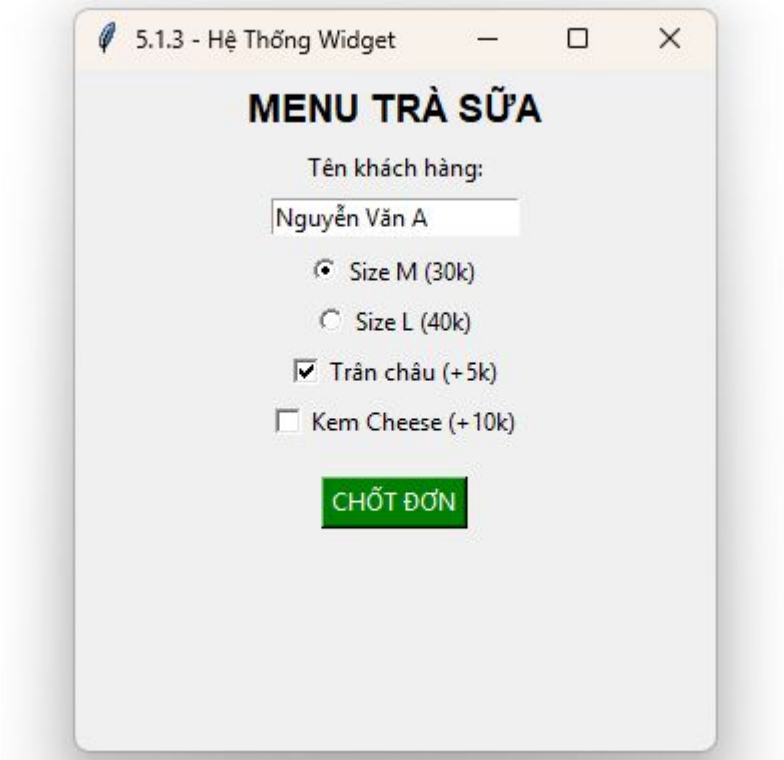
grid(row, column)

Chia mặt phẳng theo dòng cột.

place(x, y)

Định vị chính xác theo tọa độ.

5.1.4. M ức độ ưu tiên b ố cục




```python
import tkinter as tk

root = tk.Tk()
root.title("5.1.3 - Hệ Thống Widget")
root.geometry("320x340")

tk.Label(root, text="MENU TRÀ SỮA", font=("Arial", 14, "bold")).pack(pady=5)

tk.Label(root, text="Tên khách hàng:").pack()
ent_name = tk.Entry(root)
ent_name.insert(0, "Nguyễn Văn A")
ent_name.pack(pady=5)

size_var = tk.StringVar(value="M")  # Chọn 1 -> Dùng chung 1 biến StringVar
tk.Radiobutton(root, text="Size M (30k)", variable=size_var, value="M").pack()
tk.Radiobutton(root, text="Size L (40k)", variable=size_var, value="L").pack()

tc_var = tk.BooleanVar(value=True)  # Chọn nhiều -> Mỗi món 1 biến BooleanVar
riêng
cheese_var = tk.BooleanVar()
tk.Checkbutton(root, text="Trân châu (+5k)", variable=tc_var).pack()
tk.Checkbutton(root, text="Kem Cheese (+10k)", variable=cheese_var).pack()

tk.Button(root, text="CHỐT ĐƠN", bg="green", fg="white").pack(pady=15)
root.mainloop()
```

5.1.5. Cơ chế xử lý sự kiện

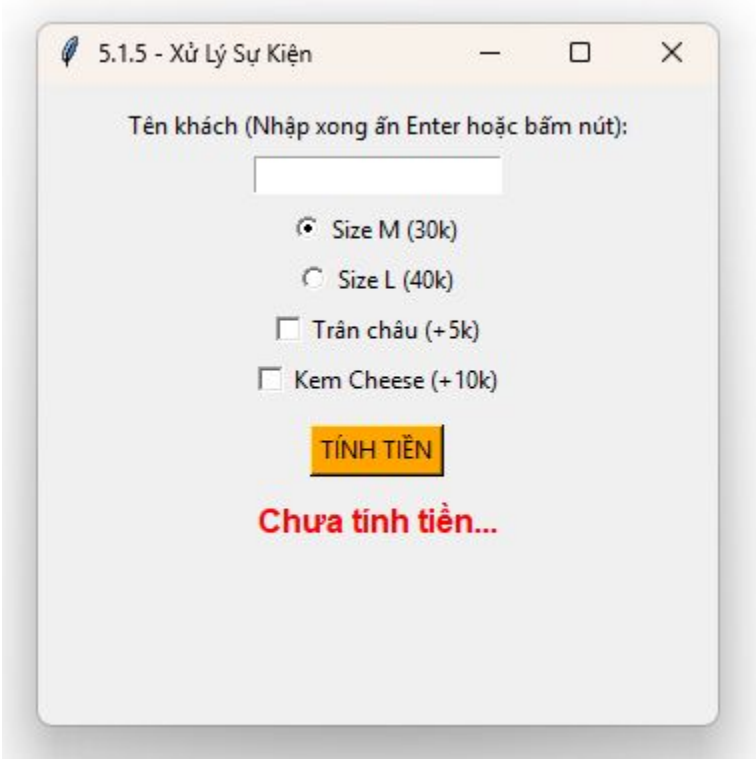
Lập trình hướng sự kiện (Event-Driven Programming): Ứng dụng ở trạng thái chờ; khi có sự kiện từ người dùng (Click chuột, ấn phím), chương trình sẽ kích hoạt hàm xử lý tương ứng (**Callback Function**).

Cách thức triển khai:

- **Nút bấm (Button):** Gắn hàm qua tham số command=ten_ham (**Lưu ý:** Không truyền dấu ngoặc tròn ()).
- **Bàn phím (Keyboard):** Dùng .bind("<Return>", ten_ham) để bắt sự kiện phím Enter.

Nguyên tắc: Tách biệt rõ ràng phần giao diện và phần logic tính toán (hàm xử lý nghiệp vụ).

5.1.5. C ơ chế xử lý sự kiện




```python
import tkinter as tk

def tinh_tien(event=None):
    ten = ent_name.get() or "Khách"
    tong = 30000 if size_var.get() == "M" else 40000
    if tc_var.get(): tong += 5000
    if cheese_var.get(): tong += 10000
    lbl_kq.config(text=f"Đơn của {ten}: {tong:,} VNĐ")

root = tk.Tk()
root.title("5.1.5 - Xử Lý Sự Kiện")
root.geometry("340x320")

tk.Label(root, text="Tên khách (Nhập xong ấn Enter hoặc bấm nút):").pack(pady=(10, 0))
ent_name = tk.Entry(root)
ent_name.pack(pady=5)
ent_name.bind("<Return>", tinh_tien)  # Bắt sự kiện ấn phím Enter

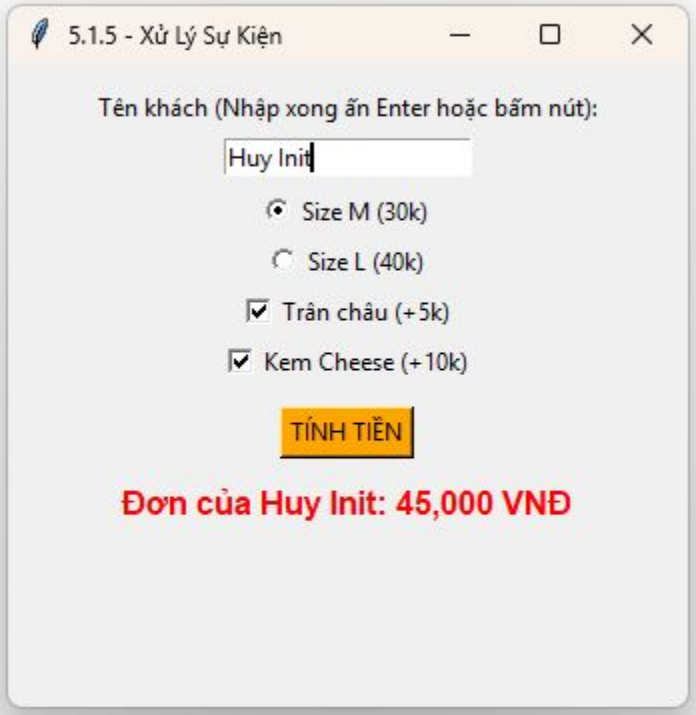
size_var = tk.StringVar(value="M")
tk.Radiobutton(root, text="Size M (30k)", variable=size_var, value="M").pack()
tk.Radiobutton(root, text="Size L (40k)", variable=size_var, value="L").pack()

tc_var, cheese_var = tk.BooleanVar(), tk.BooleanVar()
tk.Checkbutton(root, text="Trân châu (+5k)", variable=tc_var).pack()
tk.Checkbutton(root, text="Kem Cheese (+10k)", variable=cheese_var).pack()

# Chỉ khi bấm nút này hoặc ấn Enter thì hàm tinh_tien mới được gọi
tk.Button(root, text="TÍNH TIỀN", bg="orange", command=tinh_tien).pack(pady=10)
lbl_kq = tk.Label(root, text="Chưa tính tiền...", font=("Arial", 12, "bold"),
fg="red")
lbl_kq.pack()

root.mainloop()
```

5.1.5. C ơ chế xử lý sự kiện




```python
import tkinter as tk

def tinh_tien(event=None):
    ten = ent_name.get() or "Khách"
    tong = 30000 if size_var.get() == "M" else 40000
    if tc_var.get(): tong += 5000
    if cheese_var.get(): tong += 10000
    lbl_kq.config(text=f"Đơn của {ten}: {tong:,} VNĐ")

root = tk.Tk()
root.title("5.1.5 - Xử Lý Sự Kiện")
root.geometry("340x320")

tk.Label(root, text="Tên khách (Nhập xong ấn Enter hoặc bấm nút):").pack(pady=(10, 0))
ent_name = tk.Entry(root)
ent_name.pack(pady=5)
ent_name.bind("<Return>", tinh_tien)  # Bắt sự kiện ấn phím Enter

size_var = tk.StringVar(value="M")
tk.Radiobutton(root, text="Size M (30k)", variable=size_var, value="M").pack()
tk.Radiobutton(root, text="Size L (40k)", variable=size_var, value="L").pack()

tc_var, cheese_var = tk.BooleanVar(), tk.BooleanVar()
tk.Checkbutton(root, text="Trân châu (+5k)", variable=tc_var).pack()
tk.Checkbutton(root, text="Kem Cheese (+10k)", variable=cheese_var).pack()

# Chỉ khi bấm nút này hoặc ấn Enter thì hàm tinh_tien mới được gọi
tk.Button(root, text="TÍNH TIỀN", bg="orange", command=tinh_tien).pack(pady=10)
lbl_kq = tk.Label(root, text="Chưa tính tiền...", font=("Arial", 12, "bold"),
fg="red")
lbl_kq.pack()

root.mainloop()
```

5.1.6. Hộp thoại và ứng dụng mẫu

Hộp thoại chuẩn (tkinter.messagebox):

- messagebox.showwarning(): Cảnh báo lỗi dữ liệu (ví dụ: quên nhập tên).
- messagebox.askyesno(): Hộp thoại xác nhận Yes/No trước khi thực thi tác vụ quan trọng (in hóa đơn).
- messagebox.showinfo(): Thông báo kết quả thành công cho người dùng.

Mô hình ứng dụng hoàn chỉnh:

- Xác thực dữ liệu đầu vào => Tính toán logic giá tiền => Hiển thị pop-up xác nhận => Reset form chuẩn bị đón khách tiếp theo.

5.1.6. Hộp thoại và ứng dụng mẫu

```
import tkinter as tk
from tkinter import messagebox

def chot_don():
    ten = ent_name.get().strip()
    if not ten:
        messagebox.showwarning("Thiếu tên", "Vui lòng nhập tên
dán lên cốc!")
        return
    tong = 30000 if size_var.get() == "M" else 40000
    if tc_var.get(): tong += 5000
    if cheese_var.get(): tong += 10000

    bill = f"Khách: {ten}\nSize: {size_var.get()}\nTổng cộng:
{tong:,} VNĐ\n\nXác nhận chốt đơn?"
    if messagebox.askyesno("Xác nhận đơn", bill):
        messagebox.showinfo("Thành công", f"Đã in bill cho {ten}!
Chờ 3 phút nhé!")
        ent_name.delete(0, tk.END)
```

```
root = tk.Tk()
root.title("5.1.6 - App Đặt Trà Sữa")
root.geometry("340x330")

tk.Label(root, text="TIỆM TRÀ SỮA SINH VIÊN", font=("Arial", 14, "bold"),
fg="#d35400").pack(pady=10)

khung = tk.Frame(root)
khung.pack(pady=5)

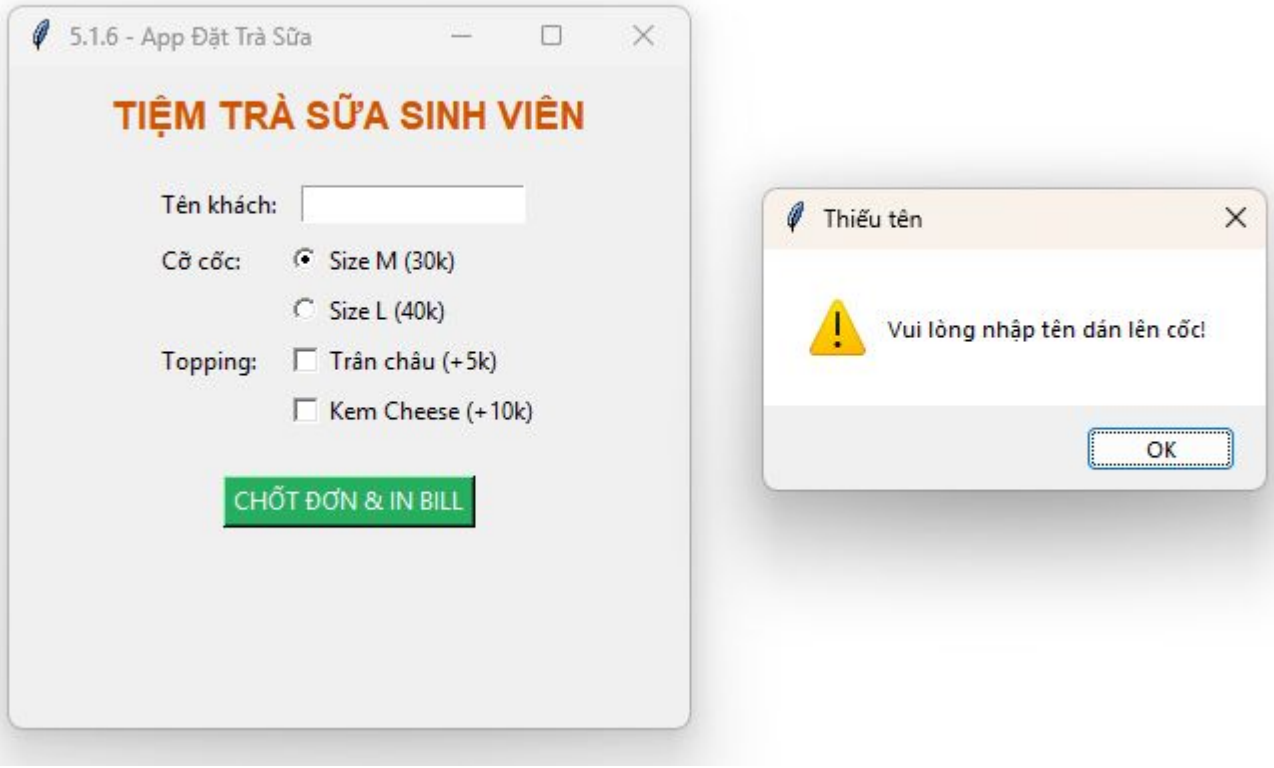
tk.Label(khung, text="Tên khách:").grid(row=0, column=0, sticky="w", pady=5)
ent_name = tk.Entry(khung, width=18)
ent_name.grid(row=0, column=1, pady=5)

size_var = tk.StringVar(value="M")
tk.Label(khung, text="Cỡ cốc:").grid(row=1, column=0, sticky="w")
tk.Radiobutton(khung, text="Size M (30k)", variable=size_var,
value="M").grid(row=1, column=1, sticky="w")
tk.Radiobutton(khung, text="Size L (40k)", variable=size_var,
value="L").grid(row=2, column=1, sticky="w")

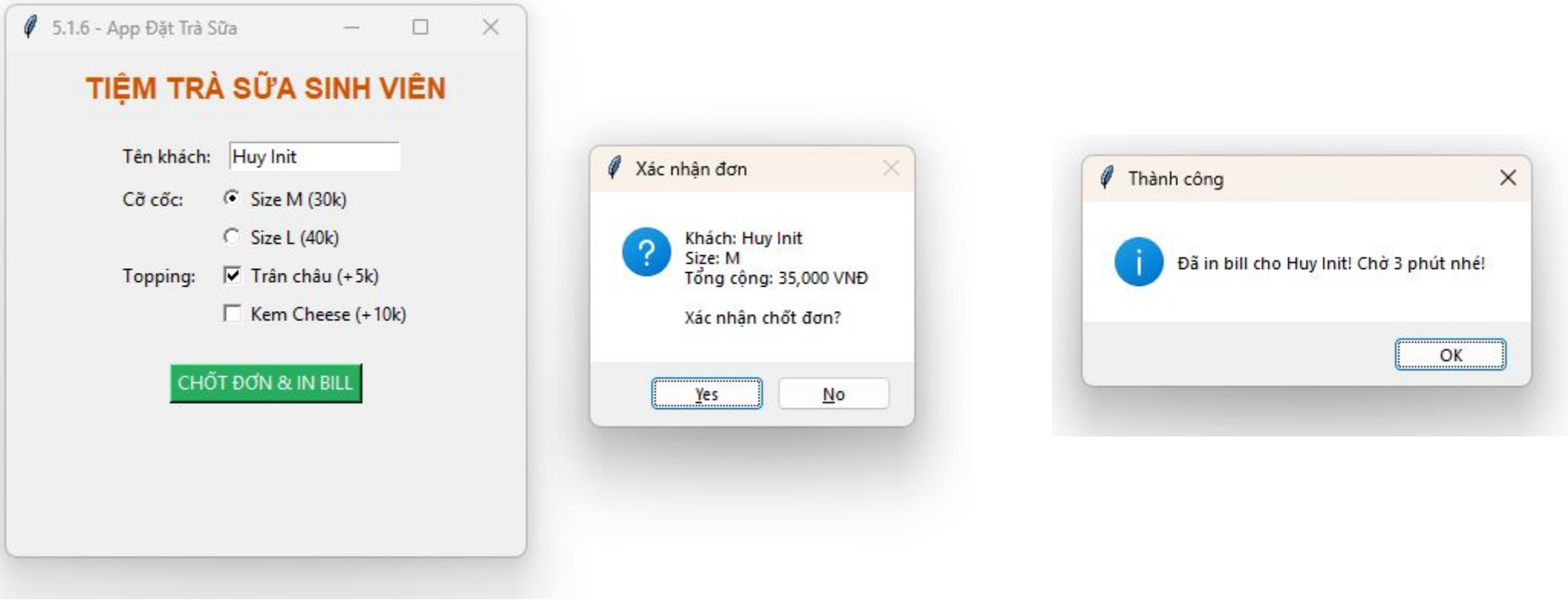
tc_var, cheese_var = tk.BooleanVar(), tk.BooleanVar()
tk.Label(khung, text="Topping:").grid(row=3, column=0, sticky="w")
tk.Checkbutton(khung, text="Trân châu (+5k)", variable=tc_var).grid(row=3,
column=1, sticky="w")
tk.Checkbutton(khung, text="Kem Cheese (+10k)",
variable=cheese_var).grid(row=4, column=1, sticky="w")

tk.Button(root, text="CHỐT ĐƠN & IN BILL", bg="#27ae60", fg="white",
command=chot_don).pack(pady=15)
root.mainloop()
```

5.1.6. H ộp tho ại và ứng d ụng m ẫu



5.1.6. Hộp thoại và ứng dụng mẫu



5.2. Lập trình Web cơ bản: Frontend/Backend & Flask

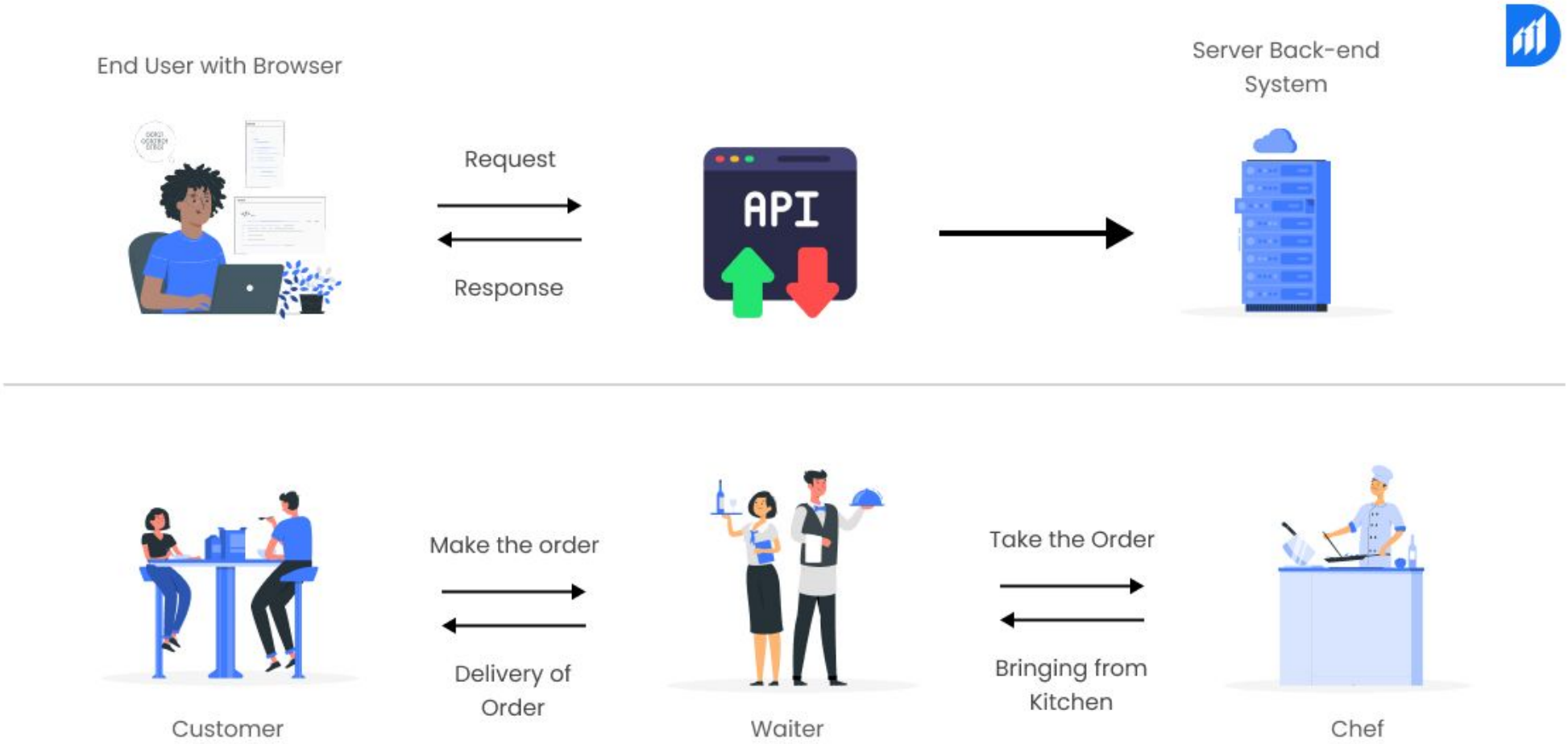
5.2.1 Mô hình Web • 5.2.2 Frontend & Backend • 5.2.3 Tổng quan Flask • 5.2.4 Định tuyến URL • 5.2.5 Template Jinja2 • 5.2.6 Biểu mẫu & Request

5.2.1. Chu trình Request – Response

Khái ni ệm quan tr ọng

1. *Front-end web development, also known as client-side development is the practice of producing some markup languages and other tools for a website or Web Application so that a user can see and interact with them directly.*
2. *Back-end development—also known as server-side development—is the architecture, code, and databases that power a website or web application behind the scenes. It focuses on the parts of the application that users don't see or interact with directly.*
3. *An API, or application programming interface, is a set of rules or protocols that enables software applications to communicate with each other to exchange data, features and functionality.*

5.2.1. Chu trình Request – Response



5.2.2. Phân biệt Frontend và Backend

Frontend (Client-side)

Vai trò: Hiển thị giao diện trực quan

Công nghệ: HTML5 CSS3 JavaScript

Môi trường: Chạy trên trình duyệt web

Mục tiêu: Tối ưu trải nghiệm người dùng

Backend (Server-side)

Vai trò: Xử lý nghiệp vụ ngầm

Công nghệ: Python Flask Django SQL

Môi trường: Chạy trên máy chủ trung tâm

Mục tiêu: Đảm bảo bảo mật hiệu năng

5.2.2. Ba trụ cột của Frontend

HTML5

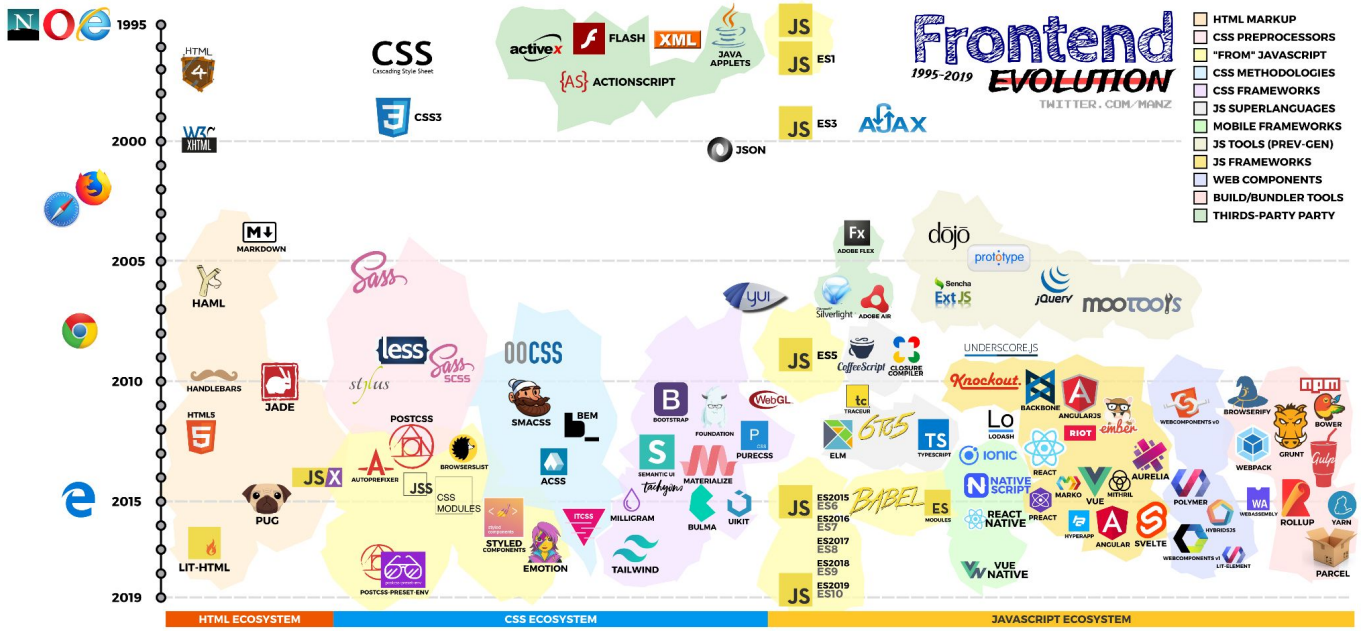
Xây dựng khung nội dung trang.

CSS3

Định dạng màu sắc bố cục.

JavaScript

Tạo tương tác động tức thời.



5.2.3. Tổng quan thư viện Flask



Micro-framework: Nhỏ gọn dễ mở rộng thư viện

Lõi nền tảng: Kết hợp bộ định tuyến Werkzeug

Jinja2 tích hợp: Hỗ trợ kết xuất trang HTML động

Khởi tạo nhanh: Chạy máy chủ chỉ vài dòng lệnh

5.2.3. Khởi tạo máy chủ Flask

```
# Cài đặt: pip install flask
```

```
from flask import Flask

app = Flask(__name__)

@app.route("/")
def trang_chu():
    return "<h1>Chào mừng đến với Blog Tâm Sự Năm
Nhất!</h1>"

if __name__ == "__main__":
    app.run(debug=True)
```

```
PS D:\phat> pip install flask
Collecting flask
  Downloading flask-3.1.3-py3-none-any.whl (103 kB)
     |████████████████████████████████| 103 kB 656 kB/s
Requirement already satisfied: click>=8.1.3 in c:\users\huyre\appdata\local\programs\python\python310\lib\site-packages (from flask) (8.
5.0)
Collecting blinker>=1.9.0
  Downloading blinker-1.9.0-py3-none-any.whl (8.5 kB)
Collecting jinja2>=3.1.2
  Downloading jinja2-3.1.6-py3-none-any.whl (134 kB)
     |████████████████████████████████| 134 kB 2.2 MB/s
Collecting werkzeug>=3.1.0
  Downloading werkzeug-3.1.9-py3-none-any.whl (228 kB)
     |████████████████████████████████| 228 kB 6.8 MB/s
Collecting markupsafe>=2.1.1
  Downloading markupsafe-3.0.4-cp310-cp310-win_amd64.whl (14 kB)
Collecting itsdangerous>=2.2.0
  Downloading itsdangerous-2.2.0-py3-none-any.whl (16 kB)
Installing collected packages: markupsafe, werkzeug, jinja2, itsdangerous, blinker, flask
Successfully installed blinker-1.9.0 flask-3.1.3 itsdangerous-2.2.0 jinja2-3.1.6 markupsafe-3.0.4 werkzeug-3.1.9
WARNING: You are using pip version 21.2.3; however, version 26.2.2 is available.
You should consider upgrading via the 'C:\Users\huyre\AppData\Local\Programs\Python\Python310\python.exe -m pip install --upgrade pip' c
ommand.
```

Flask(__name__): Khởi tạo đối tượng ứng dụng web

@app.route("/"): Gắn đường dẫn gốc với hàm

return: Trả chuỗi phản hồi về trình duyệt

app.run(): Chạy máy chủ tại cổng 5000

debug=True: Tự khởi động lại khi sửa mã

5.2.3. Khởi tạo máy chủ Flask

```python
from flask import Flask

app = Flask(__name__)

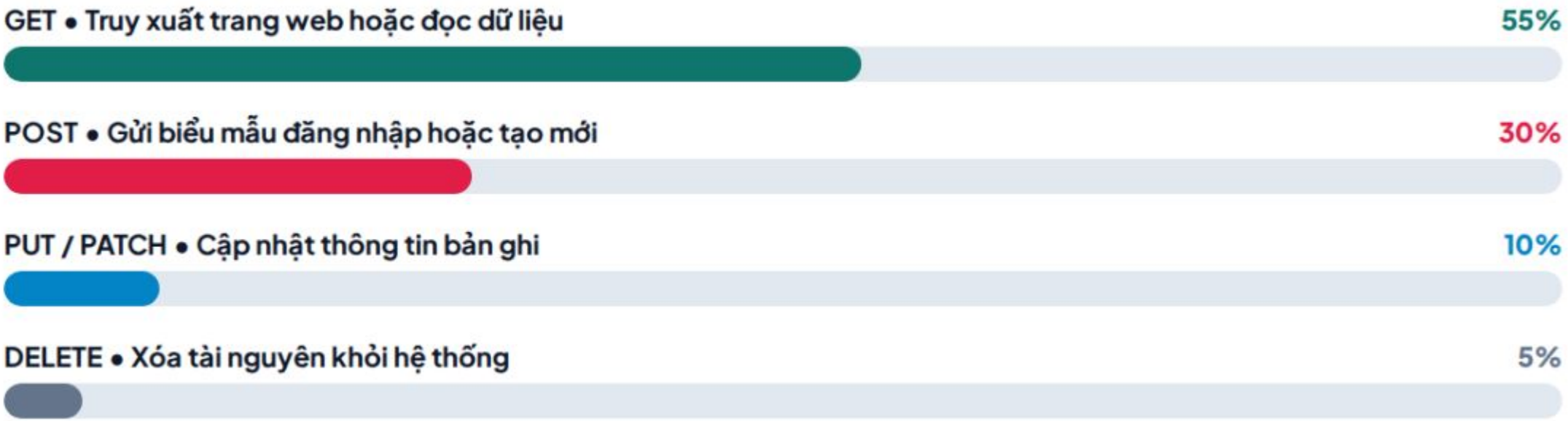
@app.route("/")
def trang_chu():
    return "<h1>Chào mừng đến với Blog Tâm Sự Năm Nhất!</h1>"

if __name__ == "__main__":
    app.run(debug=True)
```

```
PS D:\phat> python test.py
 * Serving Flask app 'test'
 * Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on http://127.0.0.1:5000
Press CTRL+C to quit
 * Restarting with stat
 * Debugger is active!
 * Debugger PIN: 874-356-493
127.0.0.1 - - [21/Oct/2026 20:29:53] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [21/Oct/2026 20:29:54] "GET /favicon.ico HTTP/1.1" 404 -
```



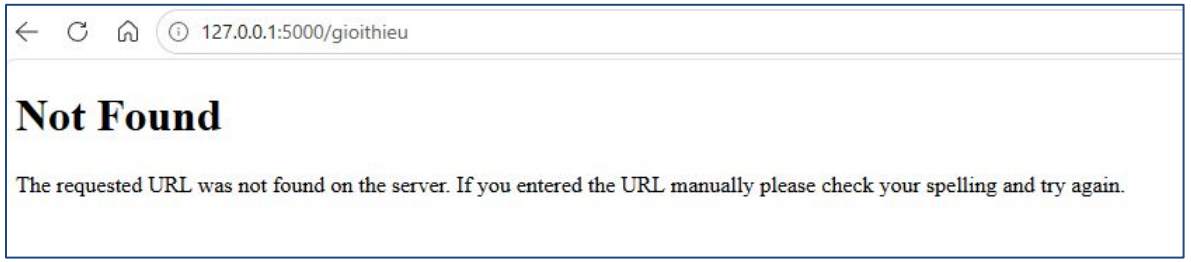
5.2.4. Các phương thức HTTP chuẩn



MỤC 5.2.4 • Định tuyến URL (Routing)








```python
from flask import Flask

app = Flask(__name__)

# Kho dữ liệu tạm thời: Danh sách các bài blog (Lưu trong List Python)
BAI_VIET = [
    "Bài 0: Ngày đầu tiên đi học đại học bị lạc đường.",
    "Bài 1: Bí kíp qua môn Lập trình Python không bị rớt.",
    "Bài 2: Cuối tháng hết tiền, ăn mì tôm vị gì ngon nhất?"
]

@app.route("/")
def trang_chu():
    return "<h2>Sảnh chính Blog</h2><p>Gõ thêm /gioi-thieu hặc /bai-viet/1
lên thanh địa chỉ nhé!</p>"

@app.route("/gioi-thieu")
def gioi_thieu():
    return "<h3>Blog này thuộc về một sinh viên năm nhất vui tính!</h3>"

# Định tuyến động: <int:id> tự động bắt con số trên URL truyền vào hàm
@app.route("/bai-viet/<int:id>")
def xem_bai(id):
    if 0 <= id < len(BAI_VIET):
        return f"<h2>Nội dung:</h2> <p>{BAI_VIET[id]}</p>"
    return "<h3>Ối! Bài viết này không tồn tại (404)!</h3>"

if __name__ == "__main__":
    app.run(debug=True)
```

MỤC 5.2.4 • Định tuyến URL (Routing)






```
 * Restarting with stat
 * Debugger is active!
 * Debugger PIN: 874-356-493
127.0.0.1 - - [21/Oct/2026 20:43:20] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [21/Oct/2026 20:46:12] "GET /gioithieu HTTP/1.1" 404 -
127.0.0.1 - - [21/Oct/2026 20:46:19] "GET /gioi-thieu HTTP/1.1" 200 -
127.0.0.1 - - [21/Oct/2026 20:46:41] "GET /gioithieu HTTP/1.1" 404 -
127.0.0.1 - - [21/Oct/2026 20:47:24] "GET //bai-viet/1 HTTP/1.1" 200 -
```

```python
from flask import Flask

app = Flask(__name__)

# Kho dữ liệu tạm thời: Danh sách các bài blog (lưu trong List Python)
BAI_VIET = [
    "Bài 0: Ngày đầu tiên đi học đại học bị lạc đường.",
    "Bài 1: Bí kíp qua môn Lập trình Python không bị rớt.",
    "Bài 2: Cuối tháng hết tiền, ăn mì tôm vị gì ngon nhất?"
]

@app.route("/")
def trang_chu():
    return "<h2>Sảnh chính Blog</h2><p>Gõ thêm /gioi-thieu hặc /bai-viet/1
lên thanh địa chỉ nhé!</p>"

@app.route("/gioi-thieu")
def gioi_thieu():
    return "<h3>Blog này thuộc về một sinh viên năm nhất vui tính!</h3>"

# Định tuyến động: <int:id> tự động bắt con số trên URL truyền vào hàm
@app.route("/bai-viet/<int:id>")
def xem_bai(id):
    if 0 <= id < len(BAI_VIET):
        return f"<h2>Nội dung:</h2> <p>{BAI_VIET[id]}</p>"
    return "<h3>Ối! Bài viết này không tồn tại (404)!</h3>"

if __name__ == "__main__":
    app.run(debug=True)
```

5.2.5. Tổ chức Template Jinja2

templates/

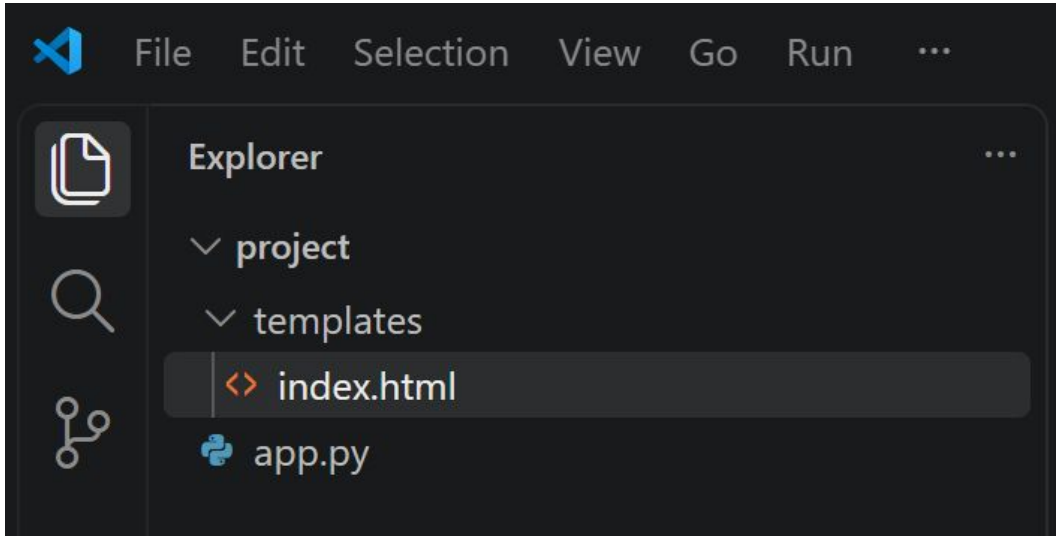
Thư mục chứa tệp HTML.

static/

Lưu trữ CSS hình ảnh tĩnh.

render_template()

Kết xuất trang kèm dữ liệu.



5.2.5. Cú pháp Jinja2 trong HTML

Cú pháp thẻ	Ý nghĩa sử dụng	Ví dụ minh họa
{{ ten_bien }}	Hiển thị giá trị biến Python	`Chào {{ user }}`
{% if ... %}	Kiểm tra nhánh điều kiện	`{% if diem ⩾ 5 %}Đạt{% endif %}`
{% for ... %}	Duyệt danh sách phần tử	`{% for mh in ds_mon_hoc %}`
{% extends %}	Kế thừa bố cục trang mẫu	`{% extends "layout.html" %}`
{# ghi_chu #}	Chú thích ẩn phía máy chủ	`{# Chú thích nội bộ #}`

5.2.5. Cú pháp Jinja2 trong HTML

```python
from flask import Flask, render_template

app = Flask(__name__)

# Mỗi bài viết là 1 Dictionary gồm Tiêu đề và Nội dung
DANH_SACH_BAI = [
    {"tieu_de": "Ngày đầu nhập học", "noi_dung": "Đi nhầm vào lớp năm tư, quê chữ ê kéo dài!"},
    {"tieu_de": "Học Flask dễ không?", "noi_dung": "Code chưa tới 20 dòng đã xong cái Blog!"}
]
@app.route("/")
def trang_chu():
    # Bưng file index.html ra và gửi kèm danh sách bài viết sang đó
    return render_template("index.html", cac_bai_viet=DANH_SACH_BAI)

if __name__ == "__main__":
    app.run(debug=True)
```

```jinja
<!DOCTYPE html>
<html>
<head>
    <title>Blog Năm Nhất</title>
</head>
<body style="font-family: Arial; max-width: 600px; margin: auto;">
    <h1>📖 Nhật ký Sinh viên Năm Nhất</h1>
    <hr>
    <!-- Dùng vòng lặp Jinja2 để tự động nhân bản các bài viết -->
    {% for bai in cac_bai_viet %}
        <div style="background: #f0f8ff; padding: 10px; margin-bottom: 10px; border-radius: 5px;">
            <h3>{{ bai.tieu_de }}</h3>
            <p>{{ bai.noi_dung }}</p>
        </div>
    {% endfor %}
</body>
</html>
```

5.2.5. Cú pháp Jinja2 trong HTML




```python
@app.route("/")
def trang_chu():
    # Bưng file index.html ra và gửi kèm danh sách bài viết sang đó
    return render_template("index.html", cac_bai_viet=DANH_SACH_BAI)
if __name__ == "__main__":
    app.run(debug=True)
```

5.2.6. Xử lý Biểu mẫu và Request

```python
from flask import Flask, render_template, request, redirect
app = Flask(__name__)

# Kho chứa bài viết ban đầu
DANH_SACH_BAI = [
    {"tieu_de": "Chào thế giới!", "noi_dung": "Đây là bài viết mở hàng của Blog."}
]
# Cho phép cả 2 phương thức: GET (vào xem) và POST (gửi bài mới)
@app.route("/", methods=["GET", "POST"])
def trang_chu():
    if request.method == "POST":
        tieu_de_moi = request.form.get("o_tieu_de")
        noi_dung_moi = request.form.get("o_noi_dung")

        # Cất bài mới vào đầu danh sách
        DANH_SACH_BAI.insert(0, {"tieu_de": tieu_de_moi, "noi_dung": noi_dung_moi})
        return redirect("/")  # Tải lại trang chủ để thấy bài vừa đăng
    return render_template("index.html", cac_bai_viet=DANH_SACH_BAI)

if __name__ == "__main__":
    app.run(debug=True)
```

```html
<!DOCTYPE html>
<html>
<body style="font-family: Arial; max-width: 600px; margin: auto;">
    <h1>✍️ Blog Tâm Sự Năm Nhất</h1>

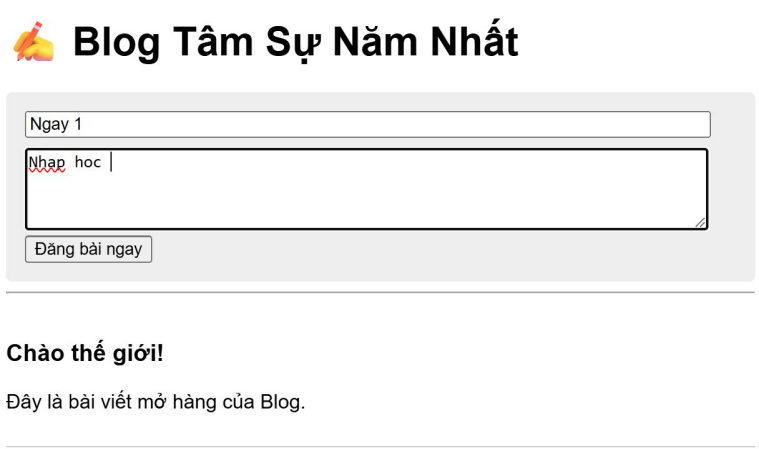
    <!-- Biểu mẫu (Form) gửi dữ liệu bằng phong bì kín POST -->
    <form method="POST" style="background: #eee; padding: 15px; border-radius: 5px;">
        <input type="text" name="o_tieu_de" placeholder="Nhập tiêu đề..." required style="width: 95%; margin-bottom: 8px;"><br>
        <textarea name="o_noi_dung" placeholder="Hôm nay bạn thế nào?" required style="width: 95%; height: 60px;"></textarea><br>
        <button type="submit">Đăng bài ngay</button>
    </form>
    <hr>

    <!-- Hiển thị danh sách bài viết -->
    {% for bai in cac_bai_viet %}
        <div style="border-bottom: 1px solid #ccc; padding: 10px 0;">
            <h3>{{ bai.tieu_de }}</h3>
            <p>{{ bai.noi_dung }}</p>
        </div>
    {% endfor %}
</body>
</html>
```

5.2.6. Xử lý Biểu mẫu và Request



5.2.6. Xử lý Biểu mẫu và Rec





5.2.6. Xử lý Biểu mẫu và Request

